```
A warehouse has 4 storage sections and 5 products.

Initially, every product has a stock level of 100 units.

Tasks
Create a 4 × 5 array where every value is 100.
Increase the stock of the first two sections by 20 units.
Reduce stock of the last two products by 10%.
Display only the middle two sections.
Display only the first three products.
Display the bottom-right 2 × 2 portion of the matrix.
```

```
  File "/tmp/ipykernel_2166/1060715943.py", line 6
    Create a 4 × 5 array where every value is 100.
               ^
SyntaxError: invalid character '×' (U+00D7)
```

Next steps: Explain error

```
import numpy as np

inventory = np.full((4, 5), 100)
print(inventory)
```

```
inventory[:2, :] = inventory[:2, :] + 20
```

```
# Reduce stock of the last two products by 10%.
inventory[:, -2:] = inventory[:, -2:] * 0.90
```

```
# Display only the middle two sections.
print(inventory[1:3, :])
```

```
[[140 140 140 115 115]
 [100 100 100  81  81]]
```

```
# Display only the first three products.
print(inventory[:,:3])
```

```
[[140 140 140]
 [140 140 140]
 [100 100 100]
 [100 100 100]]
```

```
inventory
```

```
array([[140, 140, 140, 115, 115],
       [140, 140, 140, 115, 115],
       [100, 100, 100,  81,  81],
       [100, 100, 100,  81,  81]])
```

```
# Display the bottom-right 2 × 2 portion of the matrix.

inventory[-2:,-2:]
```

```
array([[81, 81],
       [81, 81]])
```

```
A company has salary data for 4 employees:

salary = np.array([
    [30000, 5000],
    [40000, 6000],
    [50000, 8000],
    [60000, 10000]
])

Column 1 = Basic Salary
Column 2 = Allowance
```

```
Tasks
Display the complete information of the 2nd employee.
Display all basic salaries.
Display all allowances.
Increase all allowances by ₹1,000.
Increase the basic salary of the last two employees by 10%.
Display the final salary matrix.
```

```
  File "/tmp/ipykernel_2166/730193738.py", line 14
    Display the complete information of the 2nd employee.
                                                ^
SyntaxError: invalid decimal literal
```

Next steps: Explain error

```
salary = np.array([
    [30000, 5000],
    [40000, 6000],
    [50000, 8000],
    [60000, 10000]
])
```

```
# Display the complete information of the 2nd employee.

salary[1]
```

```
salary[1, :]
```

```
# Display all basic salaries.
salary[:,0]
```

```
array([30000, 40000, 50000, 60000])
```

```
# Display all allowances.
salary[:,1]
```

```
array([ 5000,  6000,  8000, 10000])
```

```
# Increase all allowances by ₹1,000.
salary[:,1] += 1000
```

```
salary[:,1] *= 1000
```

```
# salary[:,1] += 1000.5
salary[:,1] = salary[:,1] + 1000.5
```

```
# Increase the basic salary of the last two employees by 10%.

salary[-2:, 0] = salary[-2:, 0] * 1.10
```

```
print(salary)
```

```
[[   30000  6001000]
 [   40000  7001000]
 [   55000  9001000]
 [   66000 11001000]]
```

```
A startup expects its monthly users to grow from 10,000 to 100,000 over 12 months.

Tasks
Generate monthly user targets using np.linspace().
Create month numbers using np.arange().
Display users expected in Month 1 and Month 12.
Display users expected from Months 4–8.
Increase all targets by another 15%.
Display the updated targets for the last 4 months.
```

```
# Generate monthly user targets using np.linspace().
users = np.linspace(10000, 100000, 12)
```

```
print(users)
```

```
[ 10000.          18181.81818182  26363.63636364  34545.45454545
  42727.27272727  50909.09090909  59090.90909091  67272.72727273
  75454.54545455  83636.36363636  91818.18181818 100000.        ]
```

```
# Create month numbers using np.arange().

months = np.arange(1, 13)
```

```
months
```

```
array([ 1,  2,  3,  4,  5,  6,  7,  8,  9, 10, 11, 12])
```

```
# Display users expected in Month 1 and Month 12.
users[0]
```

```
np.float64(10000.0)
```

```
users[-1]
```

```
np.float64(100000.0)
```

```
# Display users expected from Months 4–8.
users[3:8]
```

```
array([34545.45454545, 42727.27272727, 50909.09090909, 59090.90909091,
       67272.72727273])
```

```
# Increase all targets by another 15%.
users = users*1.15
```

```
print(users)
```

```
[ 11500.          20909.09090909  30318.18181818  39727.27272727
  49136.36363636  58545.45454545  67954.54545455  77363.63636364
  86772.72727273  96181.81818182 105590.90909091 115000.        ]
```

```
# Display the updated targets for the last 4 months.
print(users[-4:])
```

```
[ 86772.72727273  96181.81818182 105590.90909091 115000.        ]
```

```
import numpy as np

numbers = np.array([4, 9, 16, 25])
```

```
np.sqrt(numbers)
```

```
array([2., 3., 4., 5.])
```

```
np.sqrt(49)
```

```
np.float64(7.0)
```

```
x = np.array([3, 5, 8])
y = np.array([4, 12, 15])

# distance from origin

distance = np.sqrt(x**2 + y**2)
```

```
distance
```

```
array([ 5., 13., 17.])
```

```
x = np.array([3, 5, 8])

x**3
```

```
array([ 27, 125, 512])
```

```
a = np.power(x, 3)
```

```
print(a)
```

```
[ 27 125 512]
```

```
np.cbrt(a)
```

```
array([3., 5., 8.])
```

```
a**(1/3)
```

```
array([3., 5., 8.])
```

```
numbers = np.array([-10, -5, 0, 5, 10])
np.abs(numbers)
```

```
array([10,  5,  0,  5, 10])
```

```
n = len(numbers)
for i in range(n):
  if numbers[i] < 0:
    numbers[i]*= -1

print(numbers)
```

```
[10  5  0  5 10]
```

```
np.sqrt(np.square(numbers))
```

```
array([10.,  5.,  0.,  5., 10.])
```

```
values = np.array([10.2345, 20.6789, 30.4567])

np.around(values)
```

```
array([10., 21., 30.])
```

```
values = np.array([10.2345, 20.6789, 30.4567])

np.around(values,2)
```

```
array([10.23, 20.68, 30.46])
```

```
values = np.array([2.1, 2.9, 5.5, 7.99])
#round down.

np.floor(values)
```

```
array([2., 2., 5., 7.])
```

```
values = np.array([2.1, 2.9, 5.5, 7.99])
#round up

np.ceil(values)
```

```
array([3., 3., 6., 8.])
```

```
values = np.array([2.9, 5.7, -3.8, -7.2])
np.trunc(values)
```

```
array([ 2.,  5., -3., -7.])
```

```
An e-commerce company stores order information:
```

```
orders = np.array([
    [1000, 2, 500],
    [1001, 3, 750],
    [1002, 1, 1200],
    [1003, 4, 300],
    [1004, 2, 900]
])

Columns:

Column 1 → Order ID
Column 2 → Quantity
Column 3 → Price per item


Tasks
Display the complete information of Order 1002.
Display all quantities.
Display all prices.
Display the first 3 orders.
Display the last 2 orders.
Increase the price of the last 2 products by 10%.
Increase the quantity of the first order by 1.
Display the final dataset.
Create a separate array containing only the quantity and price columns.
Display the shape and size of the dataset.
```

```
orders = np.array([
    [1000, 2, 500],
    [1001, 3, 750],
    [1002, 1, 1200],
    [1003, 4, 300],
    [1004, 2, 900]
])
```

```
# Display the complete information of Order 1002.
orders[2]
```

```
array([1002,    1, 1200])
```

```
# Display all quantities.
orders[:,1]
```

```
array([2, 3, 1, 4, 2])
```

```
orders[:,2]
```

```
array([ 500,  750, 1200,  300,  900])
```

```
# Display the first 3 orders.
orders[:3]
```

```
array([[1000,    2,  500],
       [1001,    3,  750],
       [1002,    1, 1200]])
```

```
# Display the last 2 orders.
orders[-2:]
```

```
array([[1003,    4,  300],
       [1004,    2,  900]])
```

```
# Increase the price of the last 2 products by 10%.
orders[-2:,-1] = orders[-2:,-1]*1.1
print(orders[-2:,-1])
```

```
[330 990]
```

```
# Increase the quantity of the first order by 1.
orders[0,1] += 1
```

```
print(orders)
```

```
[[1000    3  500]
 [1001    3  750]
 [1002    1 1200]
 [1003    4  300]
 [1004    2  900]]
```

```
# Create a separate array containing only the quantity and price columns.
quantity_price = orders[:, 1:3]
```

```
quantity_price
```

```
array([[   3,  500],
       [   3,  750],
       [   1, 1200],
       [   4,  330],
       [   2,  990]])
```

```
orders.shape
```

```
(5, 3)
```

```
orders.size
```

```
15
```

HW

```
A simple grayscale image can be represented as a 2D NumPy array where each number represents pixel intensity.

image = np.array([
    [10, 20, 30, 40, 50],
    [20, 30, 40, 50, 60],
    [30, 40, 50, 60, 70],
    [40, 50, 60, 70, 80],
    [50, 60, 70, 80, 90]
])
Tasks
Find the dimensions of the image.
Find its shape.
Find the total number of pixels.
Display the center pixel.
Extract the top-left 3 × 3 region.
Extract the bottom-right 2 × 2 region.
Increase the intensity of the center 3 × 3 region by 20.
Display the modified image.
```

```
A company stores quarterly revenue and expenses:

financials = np.array([
    [120, 80],
    [150, 90],
    [180, 110],
    [200, 130]
])

Rows = Q1, Q2, Q3, Q4
Columns = Revenue, Expense

Tasks
Display Q2's complete financial data.
Display all quarterly revenues.
Display all quarterly expenses.
Extract Q2 and Q3 data.
Increase Q4 revenue by 15%.
Reduce Q1 expense by 10%.
Display the final financial matrix.
```

```
A classroom has 6 rows and 5 seats per row.

Initially, all seats are empty.
```

```
0 = Empty
1 = Occupied
Tasks
Create a 6 × 5 array filled with zeros.
Mark the first seat of the first row as occupied.
Mark all seats in the 3rd row as occupied.
Mark the last two seats of the last row as occupied.
Display the first 3 rows.
Display the last 2 columns.
Display the complete seating matrix.
```